

Deep Research — cmd/server TLS cert-path defaulting fix

Revision: 1 **Last modified:** 2026-06-15T00:00:00Z **Authority:** §11.4.150 deep multi-angle web research (run in parallel with the fix per §11.4.150(F)) **Scope:** Fix (2) — the API server crashed because `config.json` omitted `tls_cert_file/` `tls_key_file` and `LoadConfig` did not backfill the `certs/` defaults. The fix backfills the default cert paths **when those cert files exist**, and otherwise surfaces a clear error.

The question

1. Is backfilling sane default cert/key paths (`certs/server.crt`, `certs/server.key`) when the config omits them the best available solution for the startup crash, vs. requiring explicit config? (ii) Are we masking a deeper problem — is **defaulting TLS cert paths a security smell** (e.g. silently serving with an unexpected/wrong cert), and does “crash → backfill default” hide a real misconfiguration the operator should see?

Angle 1 — Go stdlib `net/http ListenAndServeTLS` contract + fail-fast on missing cert (external authority)

The Go stdlib documents

`func ListenAndServeTLS(addr, certFile, keyFile string, handler Handler) error`: “files containing a certificate and matching private key for the server must be provided.” The function **always returns a non-nil error**; if the cert/key files are missing or unreadable it returns an error at startup (it does not start an insecure listener). So an empty/wrong cert path is a hard startup failure — exactly the crash the project saw when `config.json` had no cert paths and nothing backfilled them.

Best practice from the stdlib + community guidance: either (a) pass real `certFile/` `keyFile`, or (b) preload certs into `Server.TLSConfig` (via `tls.LoadX509KeyPair`) and pass empty strings to `ListenAndServeTLS`. Both require the cert material to actually resolve. A production config should also set `MinVersion: tls.VersionTLS13`, sane cipher/curve preferences, and `Read/Write/` `Idle` timeouts — orthogonal to this fix but worth noting for the server.

The fix’s design — backfill the known `certs/` defaults **only when the files exist on disk**, else emit a clear error — is the correct shape: it converts an opaque “always-non-nil-error” stdlib crash into either (a) a working default when the project’s shipped `certs/server.{crt, key}` are present (the documented project layout:

“TLS certs ... live in certs/ ... required by the API server”), or (b) an actionable error. This is **fail-fast with a helpful message**, the recommended pattern, not silent degradation.

- Go stdlib — net/http ListenAndServeTLS (cert/key required, always returns non-nil error) — <https://pkg.go.dev/net/http> (accessed 2026-06-15)
- Go stdlib — crypto/tls (LoadX509KeyPair, TLSConfig, MinVersion) — <https://pkg.go.dev/crypto/tls> (accessed 2026-06-15)
- Setting up a secure HTTPS server with crypto/tls — <https://www.slingacademy.com/article/setting-up-a-secure-https-server-in-go-with-crypto-tls/> (accessed 2026-06-15)

Angle 2 — Security: is cert-path defaulting a smell? (no-deeper-problem check, §11.4.150(C))

The legitimate worry: silently defaulting a cert path could make a server serve with an **unexpected** certificate (e.g. a stale/dev/self-signed cert) while the operator believes it is using a configured one — a security/operational smell. Two facts make this fix safe rather than smelly:

1. **Existence-gated, not blind.** The fix backfills the default path *only when the default cert files actually exist*. It is not inventing a path that then fails late, nor substituting a wrong cert over a configured one — an explicitly configured `tls_cert_file/tls_key_file` still wins (the backfill only fills the *omitted* case). The default is a documented project convention (`certs/server.crt`), not a surprise location.
2. **Loud on absence.** When the default files are absent, the fix produces a clear error instead of crashing opaquely or — the real smell to avoid — falling back to plaintext HTTP. The dangerous anti-pattern (silently downgrading TLS→HTTP on missing certs) is NOT what this fix does; “default-or-clear-error” preserves the secure-by-default posture.

The deeper problem to rule out: did the crash indicate that `LoadConfig` is generally lossy for omitted fields (i.e. other required fields could also be silently empty)? Recommendation (non-blocking, §11.4.150(C)): treat this as one instance of a general “config defaulting” responsibility — `LoadConfig` should apply documented defaults for every field that has a shipped default (cert paths, ports, timeouts), and validate-or-error for fields with no safe default. Doing it ad-hoc per field risks the next omitted field crashing the same way. This is a recommended hardening, not a flaw in the cert fix itself.

Smell that is genuinely avoided: hardcoding cert *contents* or a key path outside the repo’s controlled `certs/` dir would be a smell; defaulting to the in-repo documented `certs/` location, existence-gated, is not.

- Twelve-Factor — config defaults / explicit env separation (why omitted-field handling must be deliberate) — <https://12factor.net/config> (accessed 2026-06-15)
- Dynamically update TLS certificates without downtime (GetCertificate pattern, for the rotation follow-up) — <https://opensource.com/article/22/9/dynamically-update-tls-certificates-golang-server-no-downtime> (accessed 2026-06-15)

Finding

Best-practice-confirmed, with one recommended hardening. Existence-gated backfill of the documented certs/ default paths, with a clear error when absent and explicit config still winning, is the correct fix for the startup crash and is NOT a security smell — it preserves secure-by-default and never silently downgrades to HTTP or overrides a configured cert. Recommended follow-up (non-blocking): generalize LoadConfig so every field with a shipped default is backfilled/validated centrally, so the next omitted field cannot reproduce the same opaque crash.

Deep-research 2026-06-15: <https://pkg.go.dev/net/http>, <https://pkg.go.dev/crypto/tls>, <https://www.slingacademy.com/article/setting-up-a-secure-https-server-in-go-with-crypto-tls/>, <https://opensource.com/article/22/9/dynamically-update-tls-certificates-golang-server-no-downtime>, <https://12factor.net/config>